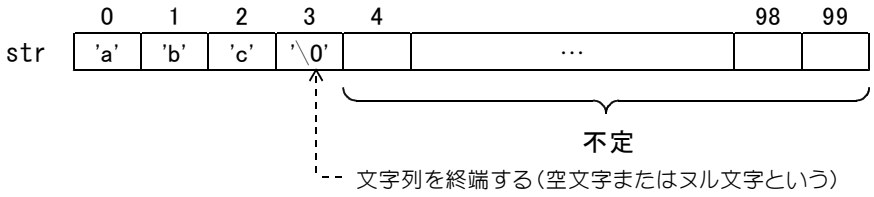
【確認１】 文字配列と文字列の入出力

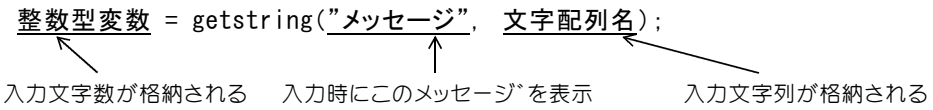
C言語の文字配列は、例えば、

```
char str[100] = "abc";
```

のように宣言します。このとき文字は最大99文字分格納でき（文字列を終端する記号 '\0' が最後に格納されるため）ますが、この例では、3文字からなる文字列"abc"で初期化しています。このとき配列の各要素には、次のように先頭から順に1文字ずつ格納され、初期化されていない要素は不定となります（何が入っているか分からない）。



また、キーボードからの文字列の入力には、getstring() という浜工独自関数を用います。getstring() は次のように使います。



ここで、キーボードから入力した文字列は第2引数に指定した文字配列に、整数型変数には入力した文字列の長さが格納されます。但し、プログラムの先頭には、以下の2行を入れておく必要があります。

```
#include <stdio.h>
#include <hamakou.h>
```

簡単なプログラムとその実行結果を以下に示します。

```
#include <stdio.h>
#include <hamakou.h>

main()
{
  char str[100];
  int  len;

  len = getstring("Input: ", str);
  printf("入力文字列 = %s, 文字列の長さ = %d\n", str, len);

  return(0);
}
```

（実行結果） Input: abcdefg↵
入力文字列 = abcdefg, 文字列の長さ = 7

また、printf() 関数の %s は、文字配列の内容を表示するときに用います。

```
                                                  整数変数
printf("入力文字列 = %s, 文字列の長さ = %d\n", str, len);
                                  文字配列
```

【確認２】１文字のみの入出力

キーボードから1文字のみを入力するには、C言語の標準関数の getchar() と、浜工独自関数の getcharacter() が使用できます。また、1文字の表示には、printf() で %c を使います。簡単な使用例を以下に示します。

```
#include <stdio.h>
main()
{
  char ch;

  printf("Input getchar: ");      // getchar()の()中にメッセージが記述でき
  ch = getchar();                 // ないので、printf()を使って表示している
  printf("入力文字 = %c\n", ch);

  return(0);
}                                 (実行結果) Input getchar: A↵
                                             入力文字 = A
```

```
#include <stdio.h>
#include <hamakou.h>
main()
{
  char ch;

  ch = getcharacter("Input getcharacter: "); // メッセージを記述できる
  printf("入力文字 = %c\n", ch);

  return(0);
}                                 (実行結果) Input getcharacter: A↵
                                             入力文字 = A
```

【確認３】文字列の長さを求める

文字配列に格納されている文字列の長さは、C言語の標準関数 strlen() で求めることができます。

但し、 strlen() を使うには、#include <string.h> が必要です。

```
整数型変数  = strlen(文字配列名 または 文字列定数)
↖                   ↑
文字数が格納される   文字数を数えたい文字列
```

【確認４】文字列のコピー

文字配列に格納されている文字列や文字列定数を、別の文字配列にコピーするには、C言語の標準ライブラリ関数 strcpy() 関数を使います。

但し、strcpy() を使うには、#include <string.h> が必要です。

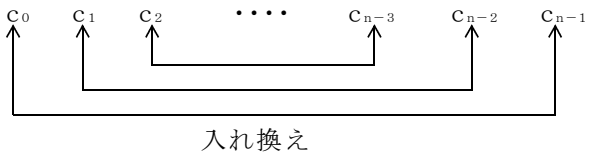
strcpy(コピー先文字配列名, コピー元文字配列名 または 文字列定数);

［例題３］

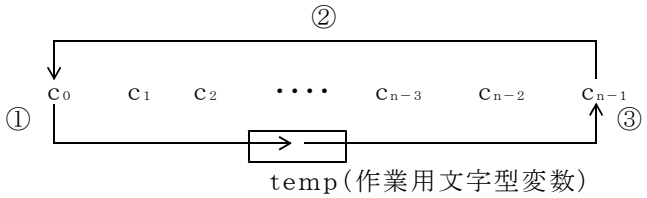
長さｎの文字列が変数 $c_0, c_1, \cdots, c_{n-1}$ に１文字ずつ入っています。これを逆順にしたものを変数 $c_0, c_1, \cdots, c_{n-1}$ に入れるアルゴリズムを考えます。

（考え方）

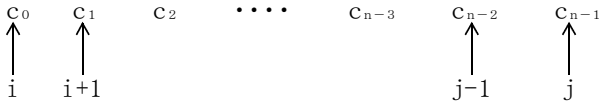
次の図のように、文字列の両端から順に、内側へ向かって前後の文字を入れ換えていきます。



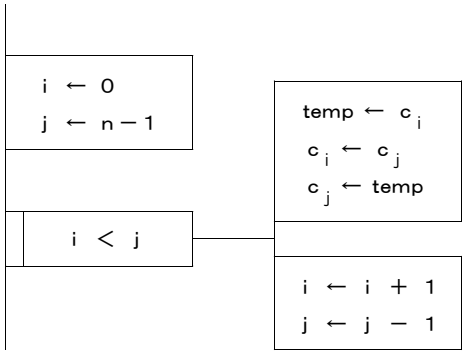
入れ換えには、作業用の変数を１つ用意し、次のように行えばよいでしょう。



また、両端から内側へ向かって入れ換えの位置を移動するには、入れ換える位置を示す変数を両端に一つずつ用意し、入れ換えのたびに、左端の位置を示す変数は＋１し、右端の変数は－１していきます。



このアルゴリズムをPADで示すと次のようになります。



［問題3.1］

次のプログラムは、［例題3］のアルゴリズムをC言語のプログラムにしたものです。空欄を埋めてプログラムを完成させなさい。

但し、文字列はキーボードより入力し、文字列の長さは255文字を越えないものとします。

(prog03-01.c)

```
#include   <stdio.h>
#include   <hamakou.h>

main()
{
    char c[ (a) ], temp;
    int  i, j, len;

    len = getstring("Input: ", c);
    printf("入力文字列 = %s\n", c);

    i = (b)            ;
    j = (c)            ;
    while (i < j) {
        (d)            ;
        (e)            ;
        (f)            ;
        (g)            ;
        (h)            ;
    }
    printf("逆順文字列 = %s\n", c);

    return(0);
}
```

（実行例）

```
Input: Hamamatsu Technical High School
入力文字列 = Hamamatsu Technical High School
逆順文字列 = loohcS hgiH lacinhceT ustamamaH
```

［問題3.2］

長さnの文字列が変数 $c_0, c_1, \cdots, c_{n-1}$ に1文字ずつ入っています。この文字列が前から読んでも後ろから読んでも同じ文字列（回文）であれば、変数pを1にし、そうでなければ0にするアルゴリズムを考え、それをpを表示するC言語プログラムにしなさい。

但し、文字列はキーボードより入力し、文字列の長さは255文字を越えないものとします。また、英大文字と英小文字は異なる文字として扱います。 (prog03-02.c)

［問題3.3］

長さnの文字列が変数 $c_0, c_1, \cdots, c_{n-1}$ に1文字ずつ入っています。この文字列中に含まれる、英大文字，英小文字，記号，数字の文字数をそれぞれ数えるアルゴリズムを考え、それを各文字種の文字数を表示するC言語プログラムにしなさい。

但し、ここでいう記号とは、英数文字と制御文字を除くすべての文字です。また、文字列はキーボードより入力し、文字列の長さは255文字を越えないものとします。

(prog03-03.c)

［問題3.4］

長さnの文字列が変数 $c_0, c_1, \cdots, c_{n-1}$ に1文字ずつ入っています。この文字列中の英大文字は英小文字に、英小文字は英大文字にし、それ以外の文字はそのままにしておくアルゴリズムを考え、それを変換後の文字列を表示するC言語プログラムにしなさい。

但し、文字列はキーボードより入力し、文字列の長さは255文字を越えないものとします。

(prog03-04.c)

（ヒント）

同じアルファベットの大文字と小文字（例えば'A'と'a'）は、文字コードの差がいくつになっているか考えなさい。文字コードの差は'A'と'a'では、'a'の方が'A'より値が大きいので、'a' - 'A' で計算できます。また、この差はどの文字でも同じになります。

［問題3.5］

長さがnの文字列が変数$a_0, a_1, \cdots, a_{n-1}$に1文字ずつ入っています。この文字列の中に、変数cに格納されている1文字と一致する文字があれば、変数pにその先頭からの文字位置を、一致する文字がなければ-1を格納するアルゴリズムを考え、pを表示するC言語プログラムにしなさい。

但し、文字列aと文字cはキーボードよりこの順序で入力し、文字列の長さは255文字を越えないものとします。また、先頭文字の位置は0とし、一致する文字が複数ある場合は、先頭に最も近い文字の位置を求めるものとします。例えば、a = "hamamatsukougyou"で c = 'u' ならば、p = 8 となります。

(prog03-05.c)

［問題3.6］

長さがnとmの2つの文字列がそれぞれ変数 $a_0, a_1, \cdots, a_{n-1}$ と $b_0, b_1, \cdots, b_{m-1}$ に1文字ずつ入っています。この2つの文字列を先頭から1文字ずつ比較し、文字が一致する部分までの先頭からの長さを求めるアルゴリズムを考え、それを一致する長さを表示するC言語プログラムにしなさい。例えば、"hamakou と "hamasyou" ならば、一致する文字列の長さは**4**になります。

但し、文字列aとbはキーボードよりこの順序で入力し、文字列の長さはどちらも255文字を越えないものとします。

(prog03-06.c)

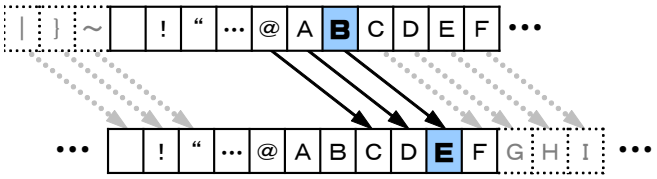
［問題3.7］（シーザー暗号）

ASCII文字集合において、' 'と'~'の範囲内（0x20≦文字コード≦0x7E）の文字と整数kが与えられたとき、この文字に対して次の操作をするものとします。ここで、｜k｜はkの絶対値を表し、また、kをキーと呼ぶことにします。

k＞0のとき　この文字をASCIIコード順で｜k｜文字後ろの文字に置き換える。
置き換える文字が'~'より後ろに来る場合は、' 'に戻って置き換える。

k＜0のとき　この文字をASCIIコード順で｜k｜文字前の文字に置き換える。
置き換える文字が' 'より前に来る場合は、'~'に戻って置き換える。

（例） ｋ＝３のとき



長さnの文字列 $c_0, c_1, \cdots, c_{n-1}$ とキーkが与えられたとき、この操作で文字列の暗号化と解読を行うC言語のプログラムを作りなさい。

例えば、キー=kで暗号化したとき、キー=－kで解読できるようにします。なお、扱う文字列とキーはこの順序でキーボードから入力し、文字列の長さは最大1023文字とします。

(prog03-07.c)

（実行例）

```
文字列: xyz
キー  : 72
abc

文字列: abc
キー  : -72
xyz

文字列: Hamamatsu Technical High School <ei21>-"Programing technology"
キー  : 8
Piuiui|{}(\mkpvqkit(Pqop([kpwwt(Dmq:9F5*Xzwoziuqvo(|mkpvwtwo"*

文字列: Piuiui|{}(\mkpvqkit(Pqop([kpwwt(Dmq:9F5*Xzwoziuqvo(|mkpvwtwo"*
キー  : -8
Hamamatsu Technical High School <ei21>-"Programing technology"

文字列: IBM
キー: -1
HAL

文字列: HAL
キー: 1
IBM
```

（ヒント）

キーを適用した後の文字が’ ’と’~’の範囲を越える際の処理は、if文などの選択構造を用いれば簡単に書くことができます。しかし、キーとキー適用後の文字コードに対して、’ ’と’~’の範囲にある文字数で割ったときの余りを考えれば、if文などを使わなくても実現できます。なお、この範囲の文字数は ’~’ － ’ ’ ＋ 1 で求められます。

レポート

次のものをE-mailで提出しなさい。

・[問題3.1]～[問題3.7]のプログラム（添付ファイル）と実行結果（本文）
・演習の感想（本文）

（提 出 先） k-kenta@hamako-ths.ed.jp ， 件名： プロ技演習３
（提出期限） ／ （ ）